Meridian

Project Title: Gauntlet Name: Meridian
Subtitle: Binary Stock Outcome Markets on Blockchain

Project Overview

Design and build a non-custodial decentralized application that enables trading of binary outcome contracts tied to the daily closing prices of MAG7 US equities (AAPL, MSFT, GOOGL, AMZN, NVDA, META, TSLA). Each contract asks a simple question — *"Will [STOCK] close above [PRICE] today?"* — and pays out $1 USDC if yes, $0 if no. Contracts expire same-day (0DTE) and settle at 4:00 PM ET using an on-chain price oracle. Users trade two complementary token types — **Yes** and **No** — on an on-chain order book. No KYC, no custody, no margin.

Problem Statement

Build a system where users can trade binary outcome tokens on whether a specific stock will close above a given strike price today. Enforce a fixed $1.00 USDC payout invariant (Yes payout + No payout = $1.00, always), automate daily market creation and settlement via a price oracle, and provide a trading interface on an on-chain order book.

Implement the full daily lifecycle:

- **Morning:** Automated creation of strike-level markets for each of the 7 stocks
- **Intraday:** Users trade Yes/No tokens on the order book
- **4:00 PM ET:** Settlement via oracle — each contract resolves to $1 or $0
- **Post-settlement:** Users redeem winning tokens for USDC

Business Context & Impact

Business Context

Binary outcome markets offer a simple, intuitive way for retail participants to express directional views on real-world events. Applying this model to daily stock price levels creates a familiar,

high-frequency product where participants answer questions like: *"Will META close above $680 today?"* The binary structure eliminates complexity — max gain is known at entry, max loss is known at entry. No Greeks, no margin calls, no unlimited downside.

Building this on a high-performance blockchain with a non-custodial architecture (user wallets, on-chain order book, oracle settlement) removes intermediaries and makes the system transparent and auditable.

Key Impact Metrics

- **Correctness:** 100% of settlements pay the correct side; the $1.00 invariant is never violated
- **Liveness:** Markets created before market open and settled within 10 minutes of market close every trading day
- **Operability:** Full lifecycle demo reproducible on testnet — create → mint → trade → settle → redeem
- **Documentation:** Clear rationale for all architecture and chain decisions; trade-offs explained

---

Technical Requirements

Preferred Languages & Frameworks

- **Smart contract:** Rust (using the Anchor framework on Solana) is preferred. Solidity on an EVM-compatible L2 is also acceptable, though EVM chains may introduce latency challenges for on-chain order book performance.
- **Frontend:** TypeScript and React (Next.js recommended)
- **Automation service:** TypeScript / Node.js

Chain Requirements

- Must deploy to a blockchain (L1 or L2) with sub-second finality — fast enough to support real-time order book trading
- Solana is the recommended chain given its transaction speed and existing order book infrastructure
- EVM L2s (e.g., Arbitrum, Base) are acceptable alternatives, but be prepared to justify how order book latency is handled
- HyperLiquid is worth considering as an alternative, though it may not natively support this type of custom instrument at this time — research and document feasibility if you explore this path
- If using Solana, deployment to **Solana devnet** is required to pass. Include reproducible scripts to deploy and run the full lifecycle on devnet.

- Use standard dev tools for your chosen chain; be prepared to justify your choices
- Secrets via environment variables; provide `.env.example`
- Never use mainnet or real funds for the core submission

Other Requirements

- Include a short risks/limitations note (no regulatory or compliance claims)
- Avoid unnecessary third-party abstractions; justify all major dependencies

---

Core Concepts

What Is a Meridian Contract?

A Meridian contract is a pair of complementary tokens — **Yes** and **No** — representing a binary outcome tied to a stock's closing price relative to a specific strike level on a given trading day.

- **Yes token:** Pays $1.00 USDC if the stock closes **at or above** the strike price. Pays $0 otherwise.
- **No token:** Pays $1.00 USDC if the stock closes **below** the strike price. Pays $0 otherwise.
- **Invariant:** Yes payout + No payout = $1.00 USDC. Always. For every contract.

Example: Contract: *"META closes above $680 today?"*

- META closes at $685 → Yes pays $1.00, No pays $0.00
- META closes at $675 → Yes pays $0.00, No pays $1.00
- META closes at exactly $680 → Yes pays $1.00, No pays $0.00 (at-or-above rule)

Pricing Relationship

Because Yes + No always equals $1.00:

- If Yes trades at $0.65, No is implicitly worth $0.35
- The Yes price approximates the market-implied probability that the stock closes at or above the strike
- Both tokens trade on the same order book (Yes token vs USDC)

What Is a CLOB?

A CLOB (Central Limit Order Book) is a trading mechanism that matches buy and sell orders based on price-time priority. Bids (buy orders) and asks (sell orders) are posted to the book, and the matching engine fills orders when a bid price meets or exceeds an ask price.

For Meridian, each strike market has **one order book** where Yes tokens are traded against USDC. You can either:

- **Use an existing on-chain CLOB** already deployed on your chosen chain (e.g., Phoenix on Solana, or similar protocols on EVM chains)
- **Build a minimal order book** as part of your smart contract (more ambitious, but demonstrates deeper understanding)

The CLOB is the trading venue. Your smart contract handles minting, settlement, and redemption; the CLOB handles price discovery and matching.

What Is an Oracle?

An oracle is a service that brings real-world data (in this case, stock prices) onto the blockchain. Your smart contract cannot natively access stock prices — it needs an oracle to provide them.

For Meridian, you need an oracle that can:

- Provide the previous day's closing price (for strike calculation each morning)
- Provide the current day's closing price at 4:00 PM ET (for settlement)
- Be read on-chain by your smart contract during the settlement transaction
- Include a staleness check (price must be recent) and a confidence/quality check

You must choose and integrate a price oracle available on your chosen chain. Popular options exist on both Solana and EVM chains. Justify your choice.

The Order Book — One Book, Two Perspectives

This is the most important architectural concept in Meridian. Each strike (e.g., "META > $680") has **exactly one order book** — the Yes token traded against USDC. But this single book serves all four user actions by flipping the perspective.

How It Works

The order book has two sides:

- **Bid side:** People wanting to buy Yes tokens (they believe the stock will close above the strike)

- **Ask side:** People wanting to sell Yes tokens (they believe the stock will close below the strike, or they are exiting a Yes position)

Because Yes + No = $1.00, selling a Yes token is economically identical to buying a No token. The order book doesn't need a separate No side — the No perspective is just the inverse of the Yes book.

The Four Trade Paths on One Book

- **Buy Yes** — The user buys Yes tokens from the ask side of the book. They now hold Yes tokens and profit if the stock closes above the strike.
- **Buy No** — The user mints a Yes/No pair (depositing $1.00 USDC), then sells the Yes token on the bid side of the book. They keep the No token. The effective cost of the No token = $1.00 − the Yes sale price.
- **Sell Yes** — The user sells their Yes tokens on the ask side of the book, receiving USDC. This closes their Yes position.
- **Sell No** — The user buys a Yes token from the ask side of the book. Since they already hold a No token, holding both Yes + No = $1.00 worth of redeemable USDC, which closes their No exposure.

Key insight: Buy Yes and Sell No are the same side of the book (both are buying Yes tokens). Buy No and Sell Yes are the same side of the book (both are selling Yes tokens). One book, four user actions, two perspectives.

UX Abstraction

To the user, the frontend presents a simple experience:

- **"Buy Yes"** and **"Sell Yes"** buttons for the bullish view
- **"Buy No"** and **"Sell No"** buttons for the bearish view

The user doesn't need to understand that Buy No is actually a mint-and-sell-Yes operation, or that Sell No is actually a buy-Yes operation. The frontend translates their intent into the correct order book action. Under the hood, it's all one book.

Position Constraints

- A user should not be able to Buy Yes if they already hold No tokens for the same strike without first selling (closing) their No position. Holding both Yes and No simultaneously is only a transient state during the mint-pair operation — it should not be a persistent user position from the trading UI.
- Similarly, a user should not Buy No if they already hold Yes tokens for the same strike without first selling their Yes position.
- The frontend should enforce this by checking the user's token balances before presenting trade options and guiding them to exit their current position first.

Underlying Assets

The 7 stocks supported in V1 (the "MAG7"):

- AAPL (Apple)
- MSFT (Microsoft)
- GOOGL (Alphabet)
- AMZN (Amazon)
- NVDA (NVIDIA)
- META (Meta Platforms)
- TSLA (Tesla)

Strike Selection

Each morning before market open, the automation service creates strikes for each stock. Strikes are placed at user-determined percentage intervals above and below the previous closing price, rounded to the nearest $10.

Algorithm

- Read previous closing price from the oracle
- Generate strikes at ±3%, ±6%, and ±9% from the previous close, rounded to the nearest $10
- This produces **6 strikes** per stock (3 above, 3 below), plus optionally the rounded previous close itself as a 7th strike
- Admin can add or adjust strikes before or during the trading day

Example: META (prev close $680)

- −9%: $620 → "META > $620"
- −6%: $640 → "META > $640"
- −3%: $660 → "META > $660"
- Close: $680 → "META > $680"
- +3%: $700 → "META > $700"
- +6%: $720 → "META > $720"
- +9%: $740 → "META > $740"

Example: AAPL (prev close $230)

- −9%: $210 → "AAPL > $210"
- −6%: $220 → "AAPL > $220"
- −3%: $220 → "AAPL > $220" *(rounds same as −6% for low-priced stocks — deduplicate)*
- Close: $230 → "AAPL > $230"
- +3%: $240 → "AAPL > $240"
- +6%: $240 → "AAPL > $240" *(same — deduplicate)*
- +9%: $250 → "AAPL > $250"

After deduplication, AAPL would have 5 unique strikes: $210, $220, $230, $240, $250.

Strikes far from the current price will trade near $1.00 (deep in-the-money) or near $0.00 (deep out-of-the-money). The most active trading happens at strikes near the current stock price.

User Stories

Buy Yes (Bullish)

- User connects wallet and sees their USDC balance
- User browses active contracts grouped by stock ticker
- User selects "META > $680" and sees current Yes/No prices
- User places a buy order (market or limit) on the order book
- Wallet prompts user to sign the transaction
- After execution, Yes token balance appears in portfolio
- Implied No price ($1.00 − Yes price) is displayed
- If user already holds No tokens for this strike, the UI prompts them to sell No first before buying Yes

Buy No (Bearish)

- Buying No is a first-class action ("Buy No" button), not a secondary workflow
- No price = $1.00 − Yes ask price
- Market order: atomic transaction that mints a Yes/No pair, immediately sells Yes at best bid — user keeps No tokens. One wallet approval.
- Limit order: atomic transaction that mints a pair, posts Yes as a limit sell at user-chosen price. User holds both tokens until the Yes sell fills.
- After execution, No token balance appears in portfolio
- If user already holds Yes tokens for this strike, the UI prompts them to sell Yes first before buying No

Sell Yes (Exit Bullish)

- User selects their Yes position and clicks "Sell Yes"
- This places a sell order for Yes tokens on the ask side of the order book
- User receives USDC when the order fills
- Portfolio updates with realized P&L (entry price vs sale price)

Sell No (Exit Bearish)

- User selects their No position and clicks "Sell No"
- Under the hood, this buys a Yes token from the ask side of the order book
- The user now holds Yes + No, which can be redeemed for $1.00, or the system handles the close automatically
- Portfolio updates with realized P&L
- To the user, this feels like simply selling their No token — the Yes-buy mechanic is abstracted away

Settlement & Redemption

- Settlement occurs automatically via the automation service at ~4:05 PM ET
- The settlement price and outcome (above/below strike) are displayed per contract
- Winning tokens show $1.00 payout; losing tokens show $0.00
- User clicks "Redeem," signs one transaction to burn tokens and receive USDC
- USDC arrives directly in the connected wallet
- Unredeemed tokens remain redeemable indefinitely

Market Maker — Mint & Quote

- Deposit $1.00 USDC per pair to mint 1 Yes + 1 No token
- Post limit orders for Yes tokens on the order book
- See exposure, fills, and P&L in a dashboard or portfolio view

---

Daily Lifecycle

- **8:00 AM ET:** Automation service reads previous close from oracle, calculates strikes
- **8:30 AM ET:** Creates contracts and order books for each strike
- **9:00 AM ET:** Markets visible on frontend, minting enabled
- **9:30 AM ET:** US market open, live trading begins
- **4:00 PM ET:** US market close
- **~4:05 PM ET:** Automation service reads oracle closing price, settles all contracts
- **4:05 PM ET+:** Redemption enabled — winners claim USDC
- **Ongoing:** Unredeemed tokens remain redeemable indefinitely

---

Smart Contract Functions

The following functions must exist in your on-chain program. Names are descriptive — use whatever naming convention fits your chain:

- **Initialize Config** — One-time setup of global configuration: admin authority, supported tickers, oracle feed references
- **Create Strike Market** — Create a single contract for one stock, one strike, one day. Includes creating the Yes/No token mints, the collateral vault, and the associated order book market. Called once per strike (not batched).
- **Add Strike** — Admin function to add extra strikes for a stock intraday
- **Mint Pair** — Any user deposits $1.00 USDC into the vault, receives 1 Yes token + 1 No token
- **Settle Market** — Reads the oracle closing price and writes the binary outcome (Yes wins or No wins) to the contract. Can only be called after 4:00 PM ET. Must validate oracle data freshness and confidence.
- **Admin Settle (Override)** — Admin-only fallback for when the oracle fails. Must enforce a time delay (e.g., 1 hour after market close) before it can be called. Used only in emergencies.
- **Redeem** — Any token holder burns their tokens and receives the payout from the vault. Winning tokens pay $1.00; losing tokens pay $0.00.
- **Pause / Unpause** — Admin can pause minting and trading in an emergency

---

Settlement Logic

- Read the oracle's closing price for the stock
- For each contract:
    - If closing price ≥ strike price → Yes payout = $1.00, No payout = $0.00
    - If closing price < strike price → Yes payout = $0.00, No payout = $1.00
- Write the outcome to the contract's on-chain account. Mark as settled. Outcome is immutable once written.

Invariants (Must Be Enforced On-Chain)

- Vault balance = $1.00 × total pairs minted (exact — fees, if any, must go to a separate account)
- Yes payout + No payout = $1.00 (at settlement, always)
- Tokens can only be created via the mint pair function
- Tokens can only be destroyed via the redeem function

- Settlement outcome is immutable once written

Oracle Integration

You must choose and integrate a price oracle. Regardless of which oracle you select, the following behaviors are required:

- **Settlement price read:** The settle function must read the stock's closing price on-chain from the oracle during the settlement transaction
- **Staleness check:** Reject prices older than a defined threshold (e.g., 5 minutes)
- **Confidence check:** Reject prices where the oracle's reported confidence band is too wide (configurable threshold)
- **Pre-market price read:** The automation service reads the previous day's close from the oracle (can be off-chain API) to calculate strikes each morning
- **Failure handling:** If the oracle is unavailable or unreliable at settlement time, the automation service retries for a defined window (e.g., 15 minutes). If still failing, admin uses the override settle function with a manual price and enforced time delay.

Frontend Application

Pages

- **Landing** — Product explanation, live prices, connect wallet call-to-action
- **Markets** — Grid of 7 stocks with live prices and active contract counts
- **Trade** — Strike list for the selected stock, order book (showing both Yes and No perspectives), Buy Yes / Buy No / Sell Yes / Sell No panel
- **Portfolio** — Active positions, settled outcomes, P&L, redeem buttons
- **History** — Trade execution log

Key UI Elements

- Contract cards showing strike, current Yes/No prices, and implied probability
- Real-time order book from the CLOB, displayed for both Yes and No perspectives (same book, two views)
- Trade panel with Buy Yes / Buy No and Sell Yes / Sell No, with position-aware constraints (can't buy Yes if holding No, and vice versa, without exiting first)
- Settlement countdown timer to 4:00 PM ET
- Simple payoff display: *"You pay $X. You win $1.00 if [STOCK] closes above [STRIKE]."*
- Portfolio with entry price, current price, P&L, and redeem button for settled contracts

---

Automation Service

An off-chain service that runs two scheduled jobs on US trading days:

- **Morning job (~8:00 AM ET):** For each of the 7 stocks, read previous close from oracle, calculate strikes, call create strike market for each. Log results. Alert on failure. Retry with backoff.
- **Settlement job (~4:05 PM ET):** For each open contract, call settle market. If oracle confidence is too wide, retry every 30 seconds for up to 15 minutes. If still failing, alert admin for manual override.

This service should live in the same repo as the smart contract and frontend.

Testing Requirements

Smart Contract Tests

- Unit tests for all core functions (mint, settle, redeem, etc.)
- Settlement logic: test at-strike, above-strike, below-strike outcomes
- Invariant tests: Yes payout + No payout = $1.00 for all possible prices
- Vault balance invariant after every mint and redeem operation
- Oracle validation: stale price, wide confidence band, valid price scenarios
- Admin override with time delay enforcement

Integration Tests

- Full lifecycle: create market → mint pair → trade on order book → settle → redeem
- All 4 trade paths: Buy Yes, Buy No, Sell Yes, Sell No
- Multi-user scenario: one user mints and quotes, another takes, both redeem

Frontend Tests

- Wallet connection flow
- Order placement and transaction signing
- Real-time price display from oracle
- Order book rendering and updates (both Yes and No views of the same book)
- Position constraint enforcement (can't hold both Yes and No simultaneously from trading)
- Portfolio and P&L accuracy

- Settlement display and redeem flow

Deployment & Success Criteria

Required: Testnet Deployment

- Deploy the full system to **Solana devnet** (or equivalent testnet for your chosen chain)
- Include reproducible scripts/commands to deploy contracts, create markets, and run the full lifecycle
- Demonstrate: create → mint → trade → settle → redeem, end-to-end on testnet
- Tests run locally and validate all key invariants
- Clear README with one-command setup (e.g., `make dev` or equivalent)

What Does Success Look Like?

- Core contract logic works reliably — minting, trading, settlement, and redemption all function correctly
- The $1.00 invariant is never violated
- All 4 trade paths functional on the order book
- Settlement executes correctly via oracle within 10 minutes of market close
- Frontend displays real-time prices, order books (both perspectives), positions, and settlement outcomes
- Position constraints enforced — users cannot hold both Yes and No tokens for the same strike from trading
- Clear, defensible trade-offs documented — architecture decisions, alternatives considered, known limitations

Bonus: Mainnet Deployment

- Deploy to **Solana mainnet-beta** (or equivalent production chain) with real infrastructure
- Funded automation wallet, production oracle feeds, production order book markets
- Monitoring and alerting for daily operations
- This is not required to pass but demonstrates production readiness

---

Glossary

- **Contract** — A binary outcome instrument: *"Will [STOCK] close above [STRIKE] today?"*
- **Yes token** — Token that pays $1.00 if the stock closes at or above the strike
- **No token** — Token that pays $1.00 if the stock closes below the strike

- **Strike** — The price level that determines the Yes/No outcome
- **Mint pair** — Deposit $1.00 USDC to create 1 Yes + 1 No token
- **Settlement** — Reading the oracle closing price and writing the binary outcome on-chain
- **Redemption** — Burning a winning token to receive $1.00 USDC from the vault
- **CLOB** — Central Limit Order Book; a matching engine that pairs buy and sell orders by price-time priority
- **Oracle** — A service that brings off-chain data (stock prices) onto the blockchain
- **Vault** — Program-owned account holding USDC collateral for each contract
- **0DTE** — Zero days to expiration; the contract expires the same day it is created